GMI German-Malaysian Institute

DIPLOMA PROGRAMME

LAB WORK 5

Academic Period: January 2026

Program	:	**CYBER SECURITY TECHNOLOGY (CBS)**
Course	:	**PENETRATION TESTING**
Code	:	**CBS 2353**
Total Marks	:	**20 MARKS**

Title	:	**WEB APPLICATION SECURITY** [Cross Site Scripting]

Student's Name	:	Abdul Hariz Bin Mohd Sarnudin
TTO's Name	:	Ts. Noormelah Binti Shamsul Anuar
Semester/Trade/Group	:	Semester 4 / DCBS / CBS 1
Date	:	28 April 2026

Assessment Specification Table (CBS)

Lab work

Raw Marks:	100
Percentage (in Doc 01):	30

No	Assessment Criteria	Full Marks	Score (if applicable)	Student's Marks	CLO	Level of Difficulty	DK, DP, NA (ETAC Programme Only)	Comments	PLO GMI: PLO CBS 9 PLO - Start Intake July 2023	PLO GMI: PLO MBOT 9 PLO
1	Understanding of Lab Objectives	4			2	3			2	2
2	Demonstration of Labwork Execution	4			2	3			2	2
3	Accuracy of Findings / Results	4			2	3			2	2
4	Fluency and Clarity of Explanation	4			2	3			2	2
5	Question & Answer (Q&A)	4			2	3			2	2
									#N/A	#N/A
									#N/A	#N/A
									#N/A	#N/A
									#N/A	#N/A
	Total Full Marks	**20**	**Total Student's Marks**							

SUMMARY

Note:
FM - Full Marks,
StM - Student's Mark

	CLO	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM	PLO GMI: PLO CBS 9 PLO - Start Intake July 2023 PLO	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM	PLO GMI: PLO MBOT 9 PLO	Raw Marks FM	Raw Marks StM	Percentage FM	Percentage StM
Total	CLO 1	0		0.0		PLO 1	0		0.0		PLO 1	0		0.0	
	CLO 2	20		6.0		PLO 2	20		6.0		PLO 2	20		6.0	
	CLO 3	0		0.0		PLO 8	0		0.0		PLO 8	0		0.0	
	CLO 4	0		0.0		#N/A	0		0.0		#N/A	0		0.0	
	CLO 5	0		0.0		#N/A	0		0.0		#N/A	0		0.0	
	CLO 6	0		0.0		#N/A	0		0.0		#N/A	0		0.0	
	Total Marks	**20**		**6.0**											

Signature

( )

NO.	CRITERIA	EXCELLENT [4]	GOOD [3]	SATISFACTORY [2]	WEAK/ NOT MET [1]	MARKS
INDIVIDUAL COMPONENT (TOTAL MARK = 20)						
1	**Understanding of Lab Objectives**	Clearly explains the purpose, scope, and objectives of the lab work	Explains objectives with minor gaps	Basic understanding of objectives	Objectives misunderstood or not explained	
2	**Demonstration of Labwork Execution**	Demonstrates lab tasks correctly, confidently, and independently	Demonstration mostly correct with minor errors	Demonstration partially correct or requires guidance	Unable to demonstrate lab tasks correctly	
3	**Accuracy of Findings / Results**	Findings are accurate, relevant, and clearly linked to the task	Mostly accurate findings with minor errors	Limited or partially correct findings	Unable to demonstrate lab tasks correctly	
4	**Fluency and Clarity of Explanation**	Explanation is clear, fluent, well-structured, and uses correct technical terms	Generally clear with minor language or structure issues	Explanation is basic and sometimes unclear	Poor, unclear, or disorganized explanation	
5	**Question & Answer (Q&A)**	Answers all questions confidently and accurately	Answers most questions correctly	Answers some questions with hesitation some questions with hesitation	Unable to answer questions correctly	
				TOTAL	20	

LABWORK 5 – CROSS SITE SCRIPTING

Labwork Requirements

1. Students must follow all procedures stated in the given lab sheet. All tasks should be completed according to the instructions provided.
2. Students are required to take clear screenshots of relevant outputs as proof of task completion. All screenshots must be properly labeled.
3. Students are required to attend a demonstration session. During the session, students must:
    - Briefly explain the labwork,
    - Perform a live demonstration, and
    - Answer questions related to the labwork.

Non-compliance with these requirements may affect the overall marks.

Submission:
Report (pdf)

Grading:

1) Understanding of Lab Objectives
2) Demonstration
3) Accuracy of Findings / Results
4) Fluency and Clarity of Explanation
5) Question & Answer (Q&A)

Team:
Individual

YOU REPORT START ON THE NEXT PAGE.

Lab - Cross Site Scripting

Objectives

In this lab, you will perform Reflected XSS and Stored XSS attacks against the DVWA (Damn Vulnerable Web Application!) at low, medium, and high security levels.

Part 1: Perform Reflective Cross Site Scripting Exploits

Part 2: Perform Stored Cross Site Scripting Exploits

Background / Scenario

In this lab, you will perform penetration tests of a web application to determine if it has been securely designed.

DVWA (Damn Vulnerable Web Application!) is a PHP/MySQL web application. It is designed to be vulnerable to common attacks to allow security professionals to test their skills and tools and students and teachers to learn and understand web application security in a legal environment.

DVWA provides four levels of security: Low, Medium, High, and Impossible. Each security level requires different skills to perform exploits. The security levels reflect different levels of security that developers may code into their applications. In this lab, you will perform exploits against three of the security levels, Low, Medium, and High, allowing you to adjust your attacks to compromise them.

Required Resources

Kali VM customized for the Ethical Hacker course

Internet access

Instructions

Part 1: Perform Reflective Cross Site Scripting Exploits

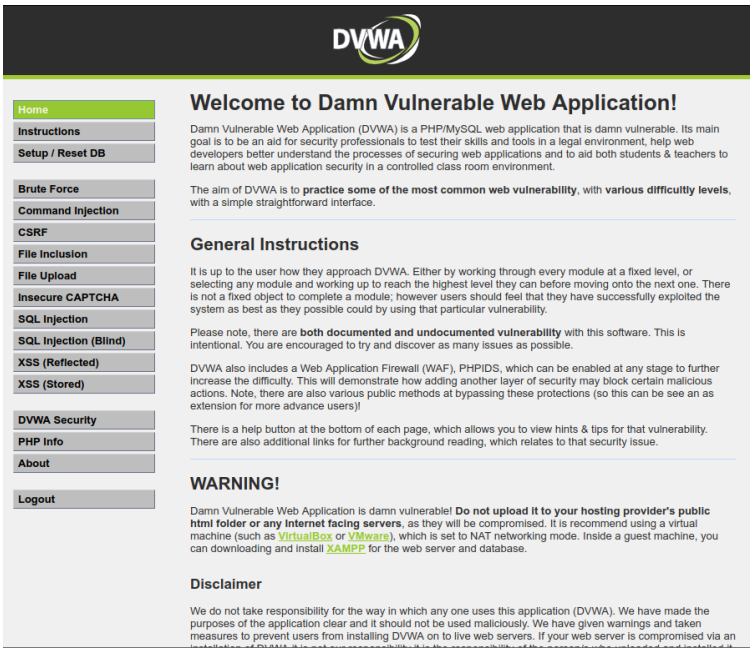
A Reflected XSS attack is one in which a malicious script is reflected off a web server to the user's browser. The script is activated through a link that the victim clicks. This will send a request to the website that has a vulnerability that enables execution of the malicious script.

Step 1: Log into DVWA.

a. From the Kali Linux VM, open a browser and navigate to the DVWA application.

b. Enter the following URL into the browser http://10.6.6.13.

c. At the login prompt, enter the credentials: **admin**/**password**.



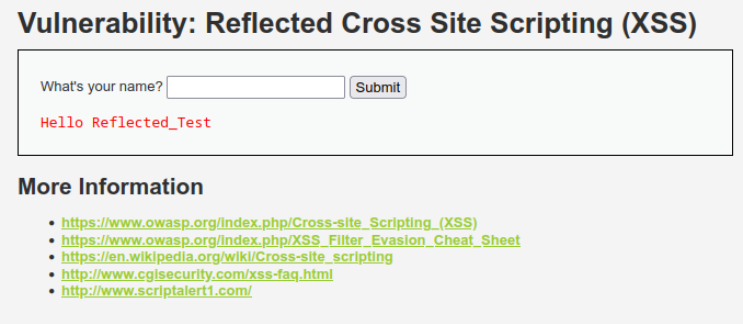
Step 2: Perform a Reflected XSS attack at Low security level.

a. Select **DVWA Security** in the left menu and select **Low** in the Security Level dropdown. Click **Submit**.

b. Select **XSS (Reflected)** from the left menu.

c. Type the string **Reflected_Test** in the **What's your name?** box and click **Submit**.

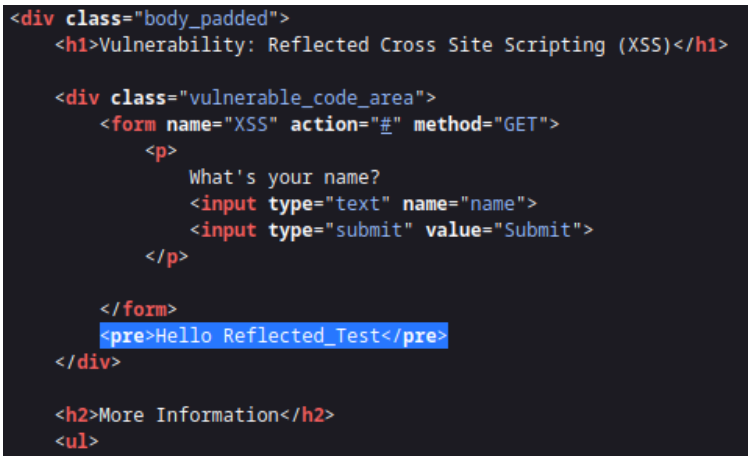
You will see the message **Hello Reflected_Test** appear.



d. Enter **CTRL+U** on the keyboard to view the source code of the page.

e. Search for the string **Hello Reflected_Test** by entering CTRL+F to open a search box.

The presence of the string in the page source HTML indicates that values entered in a user response text field are inserted into the source code for the page. This indicates to an attacker that the page may be vulnerable to reflected XSS attacks.



f. Close the source code window and return to the Reflected XSS Vulnerability page.

g. Enter the following payload in the **What's your name?** box and click **Submit**.

```
<script>alert("You are hacked!")</script>
```

An alert popup box will appear with the words **You are hacked!**. This means the site is vulnerable to Reflected XSS attacks and we have successfully exploited the vulnerability.



h. Select and copy the URL for the compromised page. Open a new browser tab and paste the URL into the URL field and press <Enter>.

http://10.6.6.13/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%22You+are+hacked%21%22%29%3C%2Fscript%3E#

You should see the same web page appear displaying the **You are hacked**! popup box. This means that if a user opens the URL a malicious script will execute. The alert box is used to simulate a malicious script in this lab.

In an ethical hacking engagement, you would try inserting a simple test script into input fields to see if the script executes. If so, the website is vulnerable to reflected XSS attacks. You could then distribute the link in a phishing attack to determine the level of security awareness among your customers' employees.

Step 3: Perform a Reflected XSS attack at Medium security level.

You will attempt the same attack, but this time the security level of the Web site will Medium.

a. Select **DVWA Security** in the left menu and select **Medium** in the Security Level dropdown. Click **Submit**.

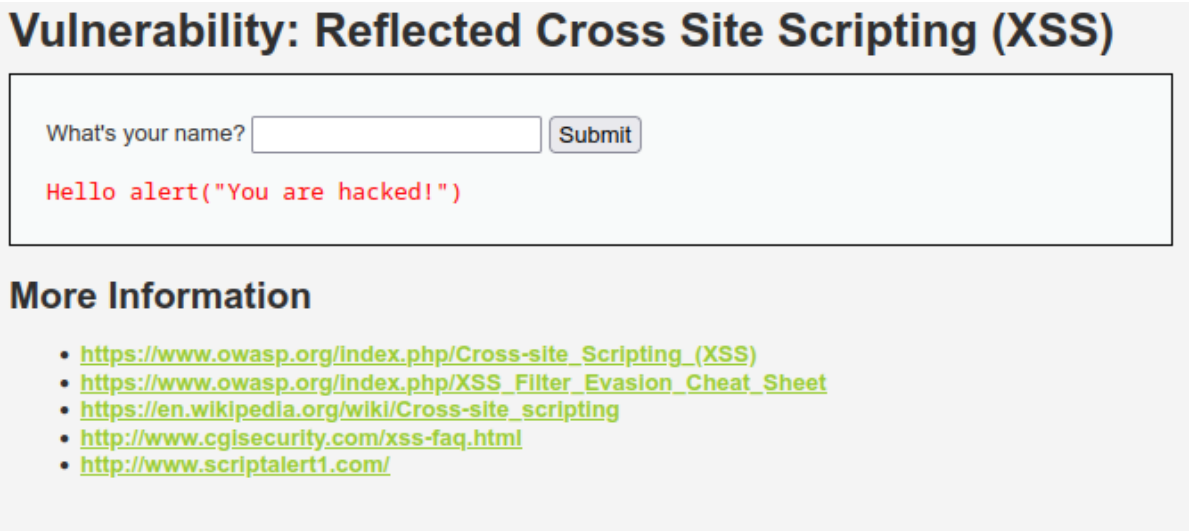
b. Select **XSS (Reflected)** in the left menu.

c. Again, enter the following payload in the **What's your name?** box and click **Submit**.

```
<script>alert("You are hacked!")</script>
```

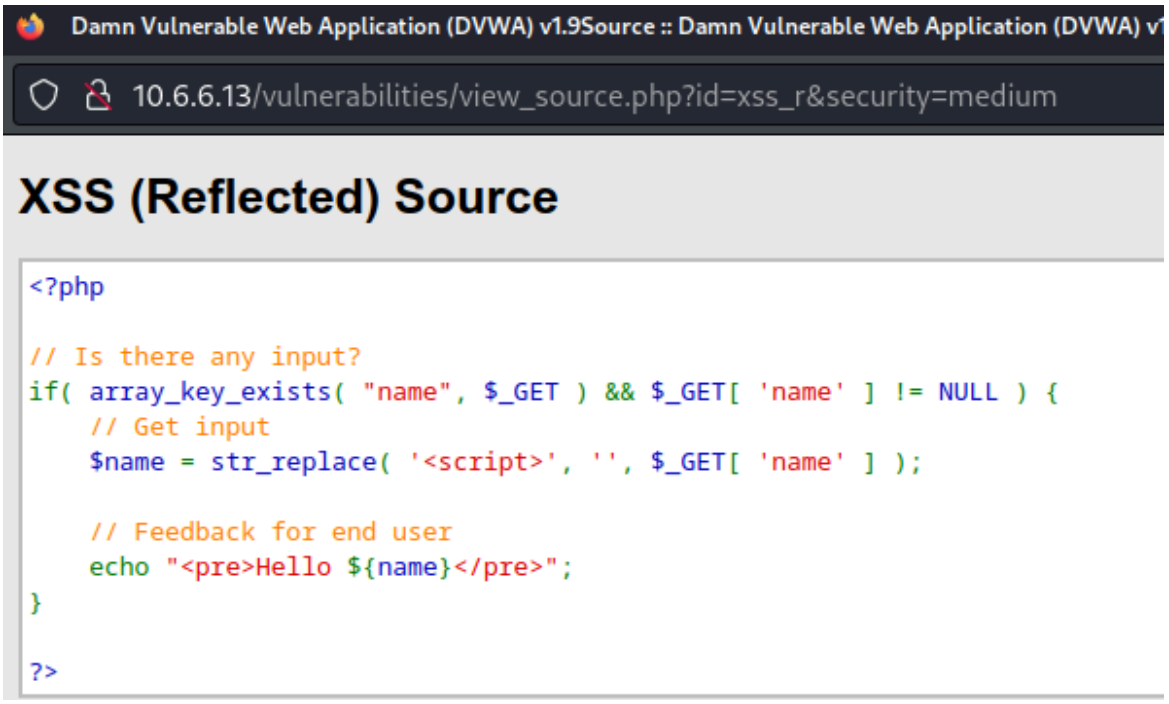
You will see a Hello response, but this time no pop up will appear. This indicates that the script did not execute. Note that the script is displayed as literal text.

We can analyze the code in the backend of the web site to investigate the reason.



d. Click the **View Source** button on the bottom right of the page and review the PHP code.

Note: On a real web server, we would not have access to this backend source code, but here on DVWS we do.



e. Note the line:

```
$name = str_replace ( '<script>', '',  $_GET[ 'name' ] );
```

This source code creates a filter, with **str_replace()** function, that removes the **<script>** tag in our payload and replaces it with a null value. This renders the payload script ineffective, so the attack failed, and no popup window is displayed. Because this script is only filtering out <script> in lower case, we can try and get around the filter by using a different tag in the payload. We will use **<ScRipt>**.

f. Close the source code window and return to the Reflected XSS Vulnerability page.

g. Enter the following payload in the **What's your name?** box and click **Submit**.

```
<ScRipt>alert("You are hacked!")</ScRipt>
```



Question:

Did the popup alert appear? If so, why?

Yes, the popup alert appeared because the str_replace() function is case-sensitive and failed to filter the mixed-case <ScRipt> tag, which the web browser still recognized and executed as a valid script.

Step 4: Perform a Reflective XSS attack at High security level.

The same attack will be attempted, but this time the security level of the website will be High.

a. Select **DVWA Security** in the left menu and select **High** in the Security Level dropdown. Click **Submit**.

b. Select **XSS (Reflected)** in the left menu.

c. Enter the following payload in the **What's your name?** box and click **Submit**. (Note the use of underscores to replace spaces.)

```
<ScRipt>alert("You_are_hacked!")</ScRipt>
```

There is a Hello message and no alert pop up box. Again, we can analyze the backend source code to investigate.

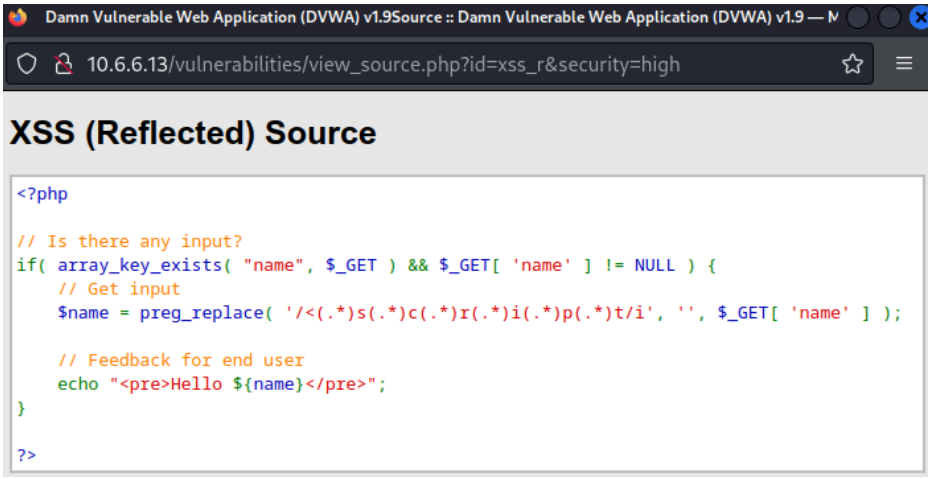


d. Click the **View Source** button and review the PHP code.

Note the following line:

```
$name = preg_replace( '/<(.*)s(.*)c(.*)r(.*)i(.*)p(.*)t/i', '', $_GET[ 'name' ] );
```

In this code, the developer used a regular expression to replace any form of the **<script>** tag, no matter what case of the characters is used, with a null value.



Question:

Which character in the script was omitted from the regular expression? How do you know?

The closing angle bracket > was omitted from the regular expression because the pattern concludes immediately after the letter 't' and does not include a literal character or wildcard to represent the end of the tag.

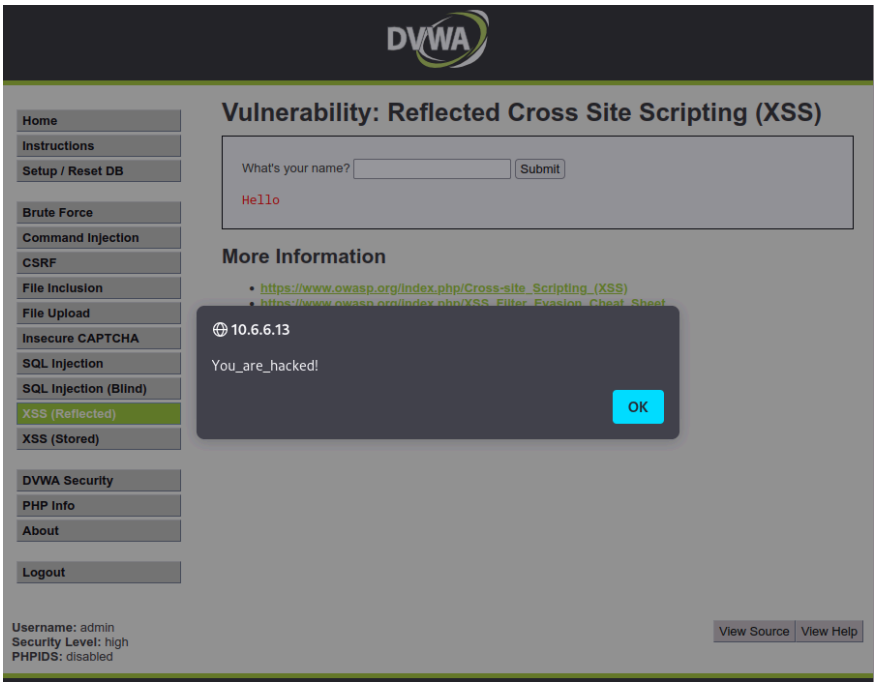
e. To bypass this filter, we must use another HTML tag instead of **<script>** to attack the site.

Close the source code window and return to the Reflected XSS Vulnerability page.

f. Enter the following payload in the **What's your name?** box and click **Submit**. (Note the use of underscores to replace spaces.)

```
<img src=x onerror=alert("You_are_hacked!")>
```

The XSS popup box will appear this time. We successfully bypassed the filter and exploited the Reflected XSS vulnerability in DVWA at High level security.



Question:

Review the text that you input into the web form. How did it work?

The payload worked because it bypassed the "script" keyword filter entirely by using an img tag that triggers the onerror event handler to execute JavaScript when the browser fails to load the invalid image source.

Part 2: Perform Stored Cross Site Scripting Exploits

With the stored XSS exploit, you enter a malicious script through user input and the script is stored on the target server in a message forum, database, visitor log, or comment field. When a user visits the target, the server exposes the user to the malicious code.

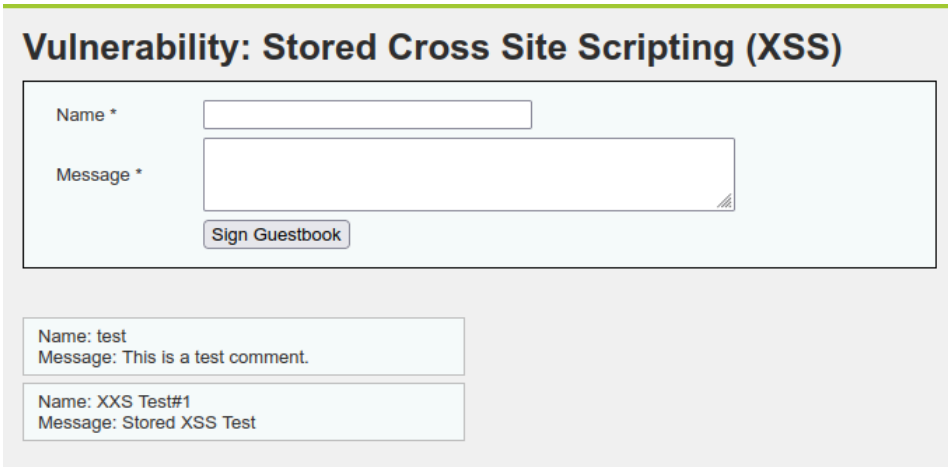
Step 1: Perform a Stored XSS attack at Low security level.

Exploiting stored XSS at low level security is easy because there are no security measures in place. You can simply submit a <script> to achieve the exploit.

a. Select **DVWA Security** in the left menu and select **Low** in the Security Level dropdown. Click **Submit**.

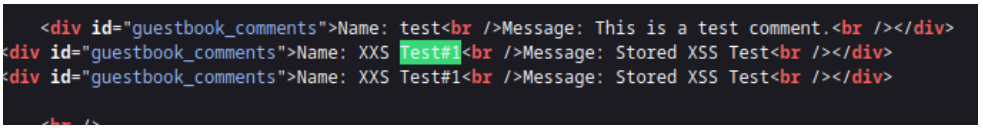
b. Select **XSS (Stored)** in the left menu.

c. Type the string **XSS Test#1** in the **Name*** field and type **Stored XSS Test** in the **Message *** field. click **Sign Guestbook**.



d. Enter **CTRL+U** on the keyboard to view the page source code. Enter **CTRL+F** to search for the **Test#1** and **Stored XSS Test** strings.

Both strings, **Test#1** and **Stored XSS Test**, will be in the page source code indicating that the two input fields may be vulnerable to a Stored XSS attack.

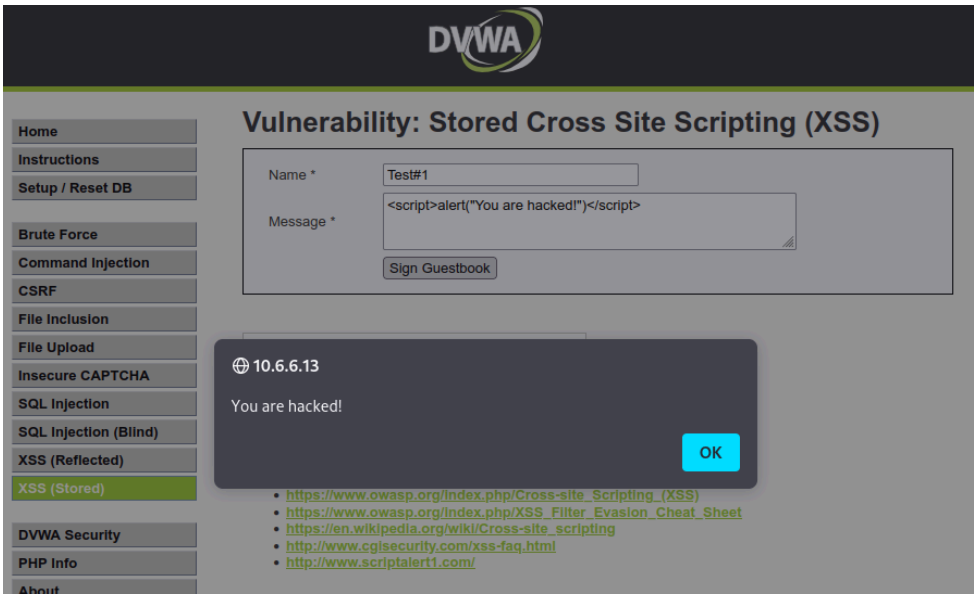


e. Close the source code window and return to the Stored XSS Vulnerability page.

f. Enter **Test#1** in the **Name *** box and enter the following payload in the **Message *** field and click **Sign Guestbook**.

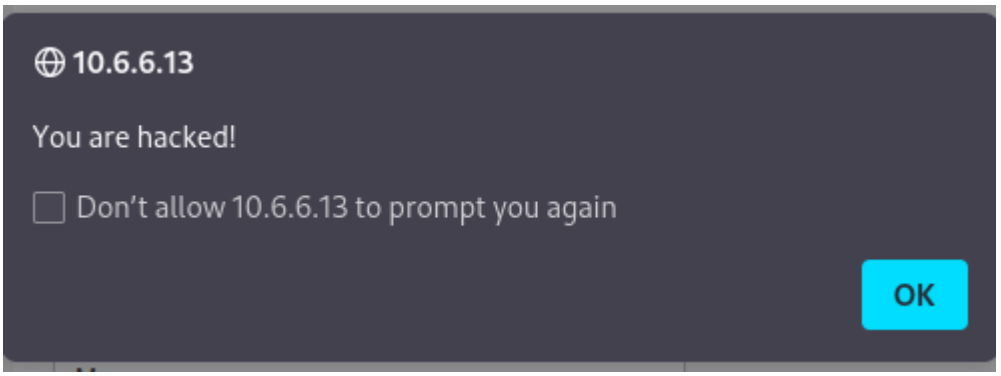
```
<script>alert("You are hacked!")</script>
```

An XSS alert popup box will appear with the words **You are hacked!**. This means the site was vulnerable to stored XSS attacks and we have successfully exploited the vulnerability.

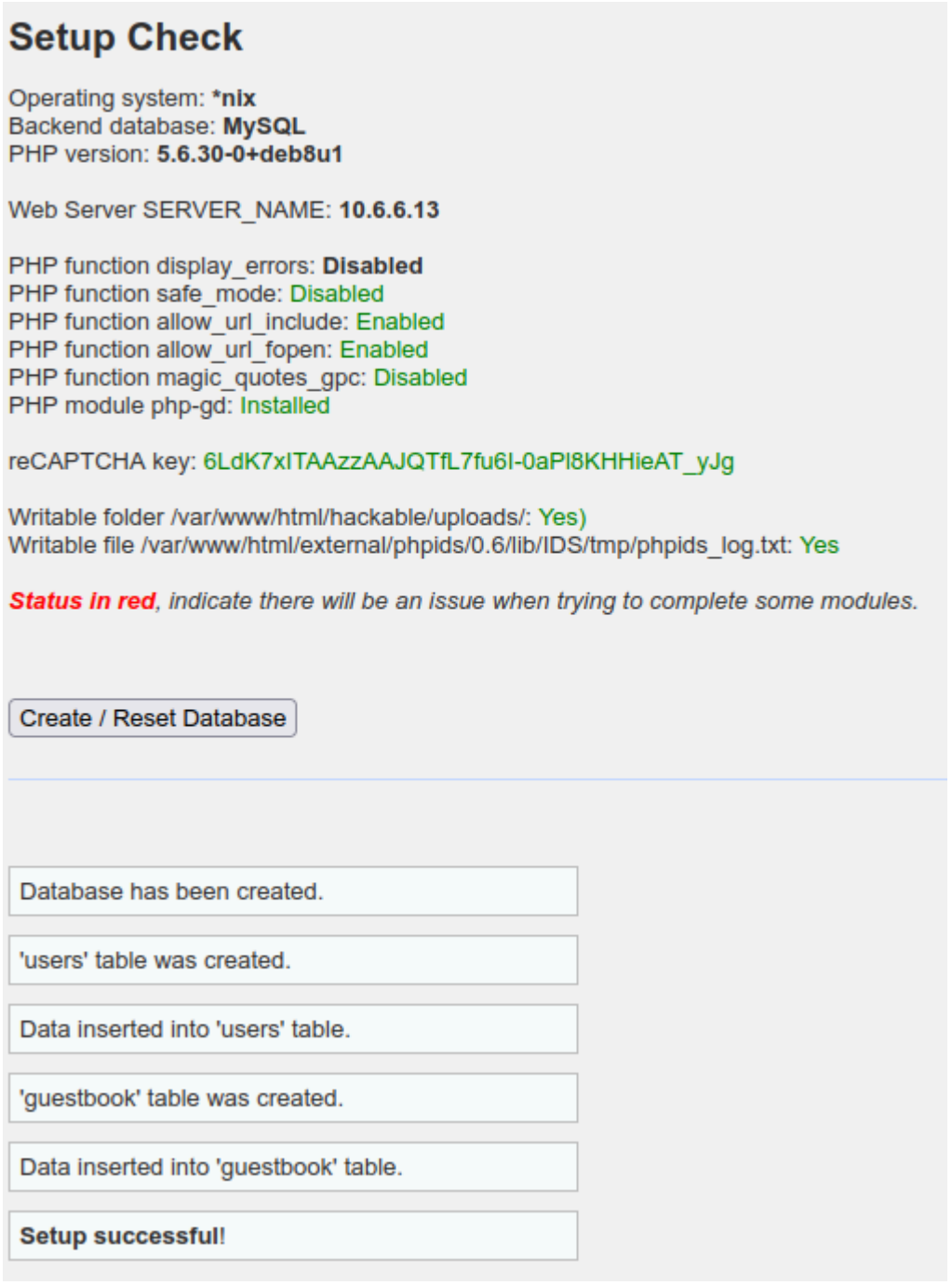


g. Refresh the page. If alerted, click **Resend** to display the page. The XSS alert popup box will appear again.

Because the XSS payload is stored in the guestbook, the alert popup box will appear each time the page is refreshed.



h. Before proceeding to the next step, clear the XSS payload from the page. Click **Setup / Reset DB** in the left menu then click **Create / Reset Database**.



Step 2: Perform a Stored XSS attack at Medium security level.

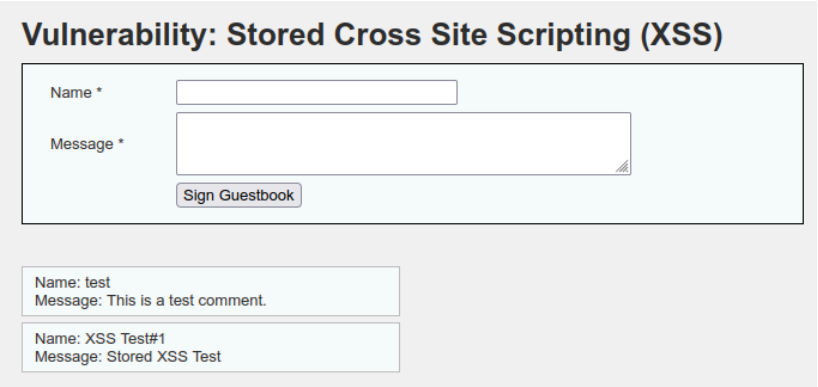
The same attack will be attempted, but this time the security level of the Web site will be changed to Medium.

Exploiting reflected XSS at medium level security is not difficult. Using <script> will be rejected but changing it to a different case such as <ScRipt> will bypass the security and achieve the exploit.

a. Select **DVWA Security** in the left menu and select **Medium** in the Security Level options dropdown. Click **Submit**.

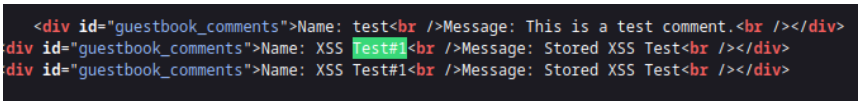
b. Select **XSS (Stored)** in the left menu.

c. Type the string **XSS Test#1** in the **Name*** field and type **Stored XSS Test** in the **Message *** field. click **Sign Guestbook**.



d. Enter **CTRL+U** on the keyboard to view the page source code. Enter **CTRL+F** to search for the **Test#1** and **Stored XSS Test** strings.

Both strings will be in the page source code indicating that the two input fields may be vulnerable to a Stored XSS attack.



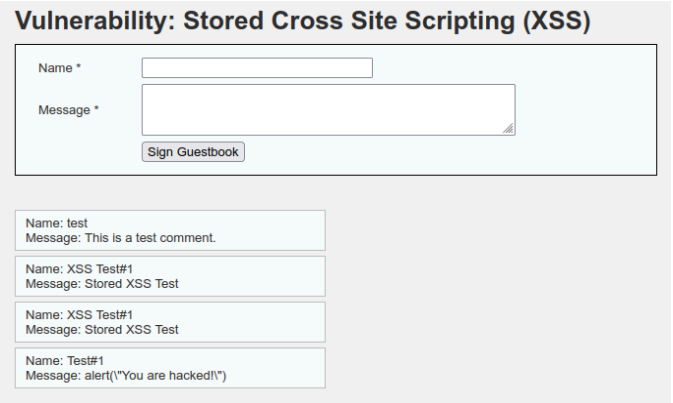
e. Close the source code window and return to the Vulnerability page.

f. Enter **Test#1** in the **Name *** box and enter the following payload in the **Message *** box and click **Sign Guestbook**.

```
<script>alert("You are hacked!")</script>
```

No popup box should appear. Refreshing the page should not cause the alert popup box to appear either.

This means that there is code in the backend that is sanitizing the user input from the **Message *** field to prevent scripts from being submitted. You can see the modified input in the last rectangle message box below the input fields.



Question:

How did the input filter script modify the input?

The <script> payload in the Message field failed because the code utilizes the strip_tags() function to remove all HTML tags and the htmlspecialchars() function to convert any remaining special characters into harmless plain text.

g. Click the **View Source** button and review the PHP source code and investigate.

You will see two blocks of code with the word **Sanitize**. The first block of code, under **// Sanitize message input**, contains two PHP functions for performing input sanitization. The **strip_tags()** function removes all html tags from the message field before storing them in the database. The **htmlspecialchars()** function converts all special characters into equivalent HTML entities so they are not reflected back in the browser.

Question:

Research the PHP addslashes() function on the internet. How did it change the input?

The addslashes() function changed the input by prepending a backslash (\) to specific characters single quotes ('), double quotes ("), backslashes (\), and NULL characters to prevent them from being interpreted as code delimiters in database queries.

The second block of code, under **// Sanitize name input**, performs input sanitation on the **Name *** field. It contains the **str_replace()** function which replaces any occurrence of the **<script>** tag with a null value. This disables the script completely.

We can attempt to bypass the security on the **Name *** field by using some other payload that does not contain **<script>** tags.

h. Close the source code window.

i. Before entering any payload into the **Name *** field, the max character length restriction of 10 characters on the field must be increased. This is a client-side setting so it is easy to change with the following steps:

1) Right-click in the **Name *** field and select **Inspect**. This opens the Web Developer Tools window and displays the page source code.

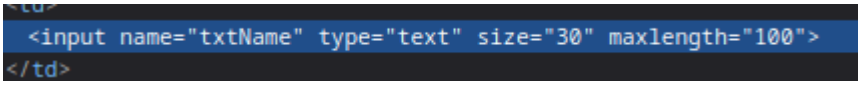
2) Find and double-click **maxlength** in the page source and change it from **10** to **100**. The maxlength property is inside the <input> tag for the text field.

3) Press **Enter** on the keyboard to apply the changes.

4) Close the Web Developer's Tools Window.

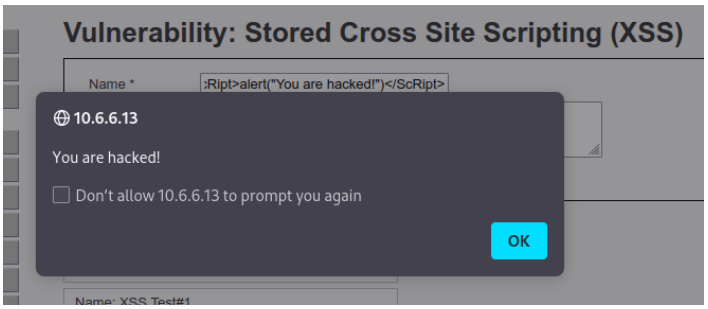
With the **maxlength** restriction changed, the XSS payload can now be entered into the **Name *** field.

Note: Changing the **maxlength** parameter does not persist. If you refresh the page, for example, the setting needs to be changed again.



j. Return to the Vulnerability page and enter the following payload in the **Name *** field.

```
<ScRipt>alert("You are hacked!")</ScRipt>
```



k. In the **Message *** field you can type any text you like and then click **Sign Guestbook**.

An XSS alert popup box will appear with the words **You are hacked!**.

Because the XSS payload is stored in the guestbook, the alert popup box will appear each time the page is refreshed or each time other users visit the page.

The popup confirms you have successfully exploited Stored XSS vulnerability at the Medium level of security.

l. Before proceeding to the next step, clear the XSS payload from the page. Click **Setup / Reset DB** in the left menu then click **Create / Reset Database**.

Step 3: Perform a Stored XSS attack at High security level.

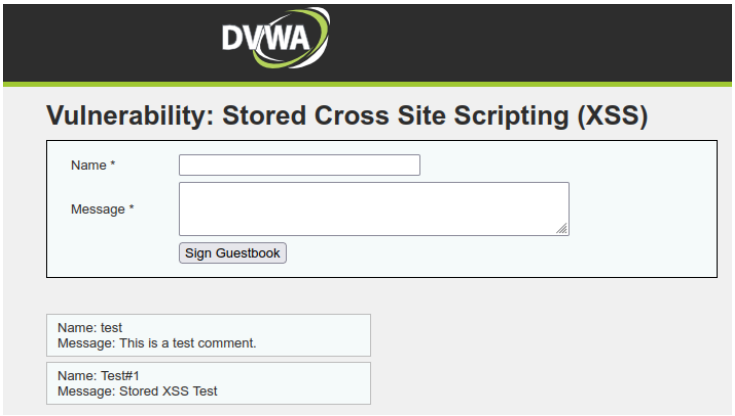
You will attempt the same attack, but this time the security level of the Web site will be set to High.

At high level the security code whitelists <scripts> and all else is rejected. However, other tags, such as svg will work.

a. Select **DVWA Security** in the left menu and select **High** in the Security Level dropdown. Click **Submit**.

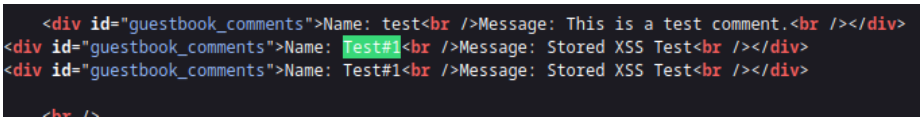
b. Select **XSS (Stored)** in the left menu.

c. Type the string **Test#1** in the **Name*** field and type **Stored XSS Test** in the **Message *** field. Click **Sign Guestbook**.



d. Enter **CTRL+U** on the keyboard to view the page source code. Enter **CTRL+F** to search for the **Test#1** and **XSS Test** strings.

Both **Test#1** and **XSS Test** should be in the page source code indicating that the two input fields may be vulnerable to a Stored XSS attack.



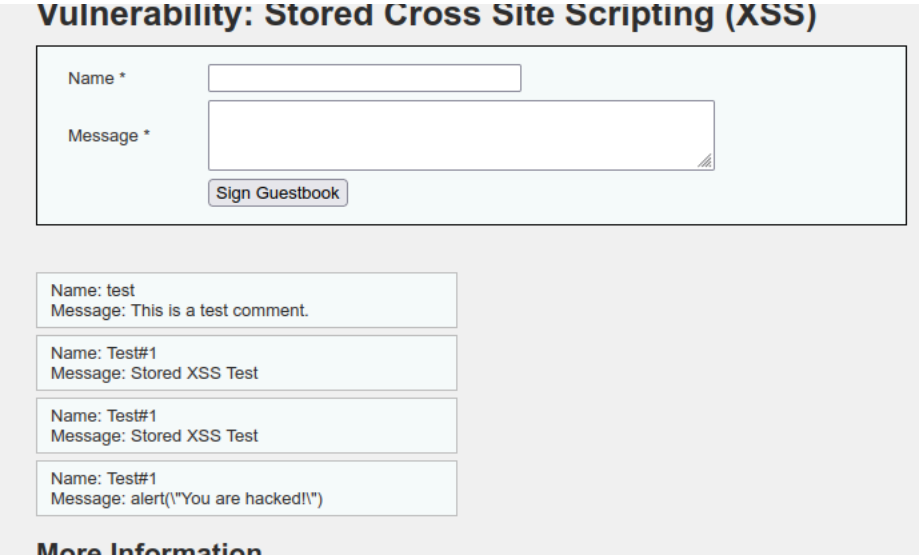
e. Close the page source code tab and return to the Vulnerability page.

f. Enter **Test#1** in the **Name *** box and enter the following payload in the **Message *** box and click **Sign Guestbook**.

```
<ScRipt>alert("You are hacked!")</ScRipt>
```

No popup box will appear. Refreshing the page will not cause the alert popup box to appear either.

This means that there is code in the site backend that is sanitizing the user input from the **Message *** field.



g. Click the **View Source** button and review the PHP source code and investigate.

You will see two blocks of code. As before with the Medium security, the first block of code, under **// Sanitize message input**, contains two php functions for performing input sanitization. The **strip_tags()** function removes all html tags from the message field before storing them in the database. The **htmlspecialchars()** function converts all special characters into equivalent HTML characters so they are not reflected back in the browser.

The second block of code, under **// Sanitize name input**, is performing input sanitation on the **Name *** field. It contains the **preg_replace()** function. This function uses a regular expression to replace any occurrence of the **<script>** tag, regardless of character case, with a null value.

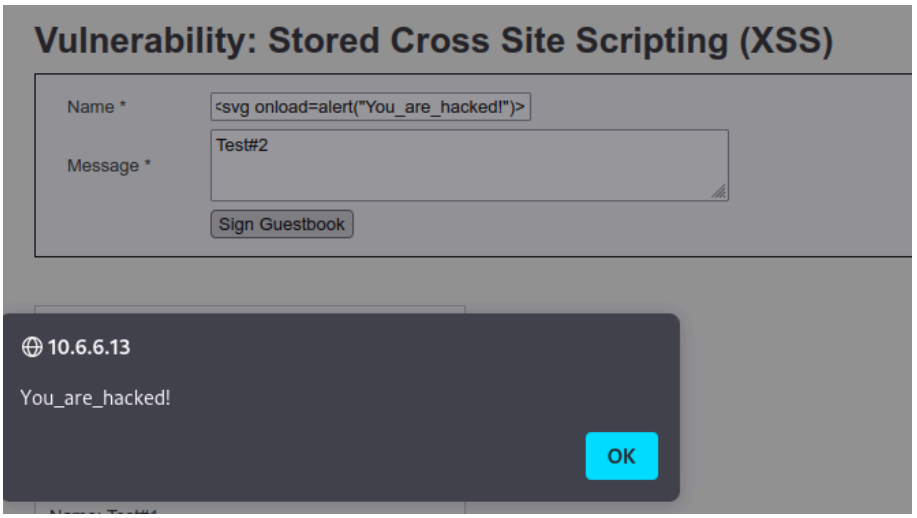
We can attempt to bypass the security on the **Name *** field by using some other payload that does not contain **<script>** tags.

h. Before entering any payload into the **Name *** field, it will be necessary to change the max character length restriction on the field, as was done above.

With the **maxlength** restriction changed, the XSS payload can now be entered into the **Name *** field.

i. Return to the Vulnerability page and enter the following payload in the **Name *** field. (Note the use of underscores to replace spaces.)

```
<svg onload=alert("You_are_hacked!")>
```



j. In the **Message *** field, you can type any text you like and then click **Sign Guestbook**.

An XSS alert popup box will appear with the message "**`You_are_hacked!`**".

Because the XSS payload is stored in the guestbook, the alert popup box will appear each time the page is refreshed.

The popup confirms you have successfully exploited a Stored XSS vulnerability at High security level.

k. Before proceeding to the next step, clear the XSS payload from the page. Click **Setup / Reset DB** in the left menu then click **Create / Reset Database**.

Step 4: Perform a stored iframe exploit.

a. Select **DVWA Security** in the left menu and select **Low** in the Security Level dropdown. Click **Submit**.

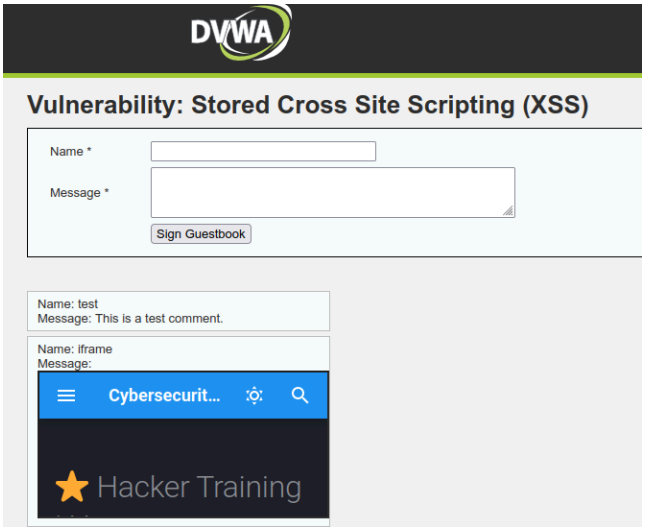
b. Select **XSS (Stored)** in the left menu.

c. Type the string **iframe** in the **Name*** field and type the following message in the **Message *** field. click **Sign Guestbook**.

```
<iframe src="http://h4cker.org"></iframe>
```

The H4cker website should now be displayed under the iframe test message.

This is a powerful exploit because the threat actor could send the browser to a malicious website.



d. Before proceeding to the next step, clear the XSS payload from the page. Click **Setup / Reset DB** in the left menu then click **Create / Reset Database**.

Step 5: Perform a stored cookie exploit.

Stealing the cookies of website visitors has security implications. Cookies contain information about how and when users visit a web site and sometimes authentication information, such as usernames and passwords. Without proper security measures, a threat actor can capture cookies and use them to impersonate specific users and gain access to their information and accounts.

a. Select **DVWA Security** in the left menu and select **Low** in the Security Level options dropdown. Click **Submit**.

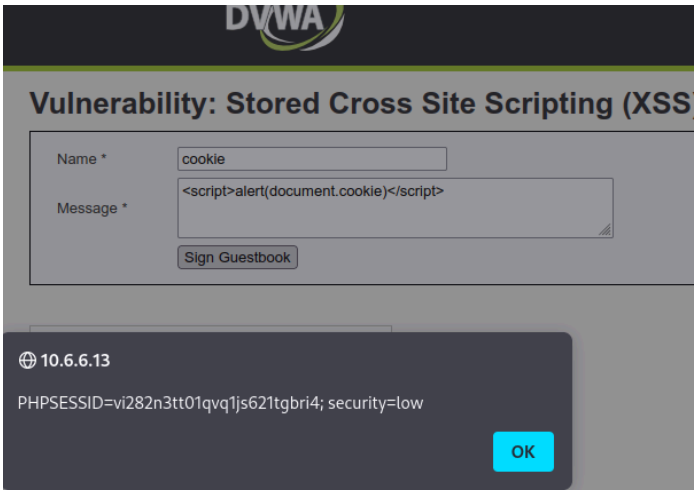
b. Select **XSS (Stored)** in the left menu.

c. Type the string **cookie** in the **Name*** field and type the following message in the **Message *** field. click **Sign Guestbook**.

```
<script>alert(document.cookie)</script>
```

A popup box with the cookie will be presented. This is a cookie that PHP uses to keep of track of running sessions.

An exploit could modify the XSS script to have the cookie sent to another destination rather than just displaying it.



d. Click **OK** in the pop-up box.

e. Try to steal cookies at the medium and high security levels using techniques that you have learned in this lab.

Question

Reflection Question

When conducting an ethical hacking test of a client web application, what does it tell you if the application is vulnerable to XSS at the low, medium, or high level?

The vulnerability level indicates the maturity of the client's security posture, revealing whether they lack basic output encoding (Low), rely on easily bypassed blacklists and case-sensitivity filters (Medium), or have implemented more complex but flawed regex-based protections that fail to account for alternative HTML event handlers (High).

-THE END-